

Today 9:19 AM

Here is my Pygame arm-wrestling game (code attached). Pressing Left/Right arrows moves the arm toward the computer's side instead of mine. The player wins at `arm_position <= - self.target_limit`. Find the line causing this, explain why, and give the minimal fix.

****# Arm Wrestle Showdown Lab****

This project is a tug-of-war button mashing game using ******Pygame******. It introduces students to vector interpolation, resource/stamina management, alternating key-stroke detection, and simple AI pressure modeling inside an object-oriented codebase.

\---

****## What's Provided****

A working Arm Wrestling game with:

\- An arm wrestling table arena rendering procedural arms, elbows, shoulders, and clasping hands

\- An alternating input mechanism requiring rhythmic pressing of Left and Right arrow keys

\- A stamina bar that depletes during rapid pressing and recovers naturally over time

\- Continuous computer AI force pushing toward the player's side

\- Win/loss boundary detection and a post-game rematch screen

It has ******one deliberate bug****** and ******three optional features****** left as tasks to implement. You are expected to ******analyze******, ******interact with an AI assistant******, and ******complete/fix****** the game to make it fully functional and more interesting.

**** **Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.****

\---

*** Getting Started**

**** Setup**

1\. Make sure you have Python 3.10+ installed.

2\. Install dependencies:

\`\`bash

pip install pygame

\`\`

3\. Run the game:

\`\`bash

python main.py

\`\`

****Controls:**** Rapidly alternate Left Arrow and Right Arrow to push, R to restart after a match.

*** Tasks to Complete**

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

**** Task 1: Fix the inverted arm push bug**

Alternating Left and Right arrow keys pushes the arm toward the computer's side instead of pulling it toward the player's side,

causing the player to pin themselves. Ensure that player keystrokes push the hands toward the player's winning threshold.

**** Task 2: Implement dynamic AI surge**

The computer currently applies force at a predictable constant rate. Introduce an AI stamina cycle where the computer periodically enters a brief high-power surge with increased pushing force, followed by a cooldown period with reduced resistance

**** Task 3: Implement an exhaustion warning indicator**

Building upon Task 2's surge cycle, give the player visual cues for fatigue states. Display an active warning indicator when the AI enters its power surge, and show an exhaustion state whenever the player's stamina drops too low to push

**** Task 4: Implement counter-surge bonus resistance**

Building upon the warnings in Task 3, reward strategic timing during fatigue states. If the player pushes right as the AI's surge ends and enters its cooldown phase, grant a temporary stamina recovery boost and double push strength to mount a comeback.

\---

*** Expected Behavior**

\- Rapidly alternating between Left Arrow and Right Arrow pulls the hands toward the player's side.

\- Rapid pushing drains stamina; falling below 10 stamina temporarily halts input until it recovers.

\- Reaching -100.0 awards victory to the player, while reaching +100.0 triggers a computer win.

\- Pressing R on the Game Over screen resets stamina, arm position, and game states for a rematch.

\---

****## Folder Structure****

\\\`

arm_wrestle/

|— game/

| |— game_engine.py

|— main.py

|— README.md

\\\`

\---

****## Submission Checklist****

Submission is only the following three things:

\- [] A 10-second video of gameplay ******before****** your changes, showing the bug/broken behavior

\- [] A 10-second video of gameplay ******after****** your changes, showing the bug fixed and the new features working

\- [] The Chat/LLM used page link, with the complete chat history
gameengine.py

import math

import random

import pygame

class GameEngine:

```
def __init__(self, width, height):  
    self.width = width  
    self.height = height  
  
    self.arm_position = 0.0  
    self.target_limit = 100.0  
    self.last_key = None  
  
    self.stamina = 100.0  
    self.max_stamina = 100.0  
  
    self.winner = None  
    self.game_state = "PLAYING"  
    self.ai_strength = 0.35
```

main.py

import sys

import pygame

from game.game_engine import GameEngine

SCREEN_WIDTH = 600

SCREEN_HEIGHT = 620

FPS = 60

def main():

pygame.init()

pygame.display.set_caption("Arm Wrestle Showdown")

```
screen = pygame.display.set_mode((SCREEN_WIDTH,  
SCREEN_HEIGHT))
```

```
clock = pygame.time.Clock()
```

```
engine = GameEngine(SCREEN_WIDTH, SCREEN_HEIGHT)  
  
running = True  
while running:  
    for event in pygame.event.get():  
        if event.type == pygame.QUIT:  
            running = False  
        else:  
            engine.handle_event(event)  
  
    engine.update()  
    engine.render(screen)  
    pygame.display.flip()  
    clock.tick(FPS)
```

```
if __name__ == "__main__":
```

```
    main()
```

👍 The bug is in **both** key handlers inside `handle_event()` .

Problematic lines

</> Python

```
self.arm_position += 4.2
```

They appear here:

</> Python

```
if event.key == pygame.K_LEFT:
    if self.last_key != pygame.K_LEFT:
        self.arm_position += 4.2    # BUG
```

and again:

</> Python

```
elif event.key == pygame.K_RIGHT:
    if self.last_key != pygame.K_RIGHT:
        self.arm_position += 4.2    # BUG
```

Why this causes the problem

Your game defines the directions like this:

</> Python

```
if self.arm_position <= -self.target_limit:
    self.winner = "PLAYER"
```

So:

- **Negative position** → player's side → player wins
- **Positive position** → computer's side → computer wins

But both Left and Right currently do:

</> Python

```
self.arm_position += 4.2
```

`+=` increases the position, so every valid player keypress moves the arm toward **+100**, which is the **computer's winning side**.

Minimal fix

Change both `+=` operations to `-=` :

</> Python

```
if event.key == pygame.K_LEFT:
    if self.last_key != pygame.K_LEFT:
        self.arm_position -= 4.2
        self.stamina = max(0.0, self.stamina - 2.0)
        self.last_key = pygame.K_LEFT
```

```
elif event.key == pygame.K_RIGHT:
    if self.last_key != pygame.K_RIGHT:
        self.arm_position -= 4.2
        self.stamina = max(0.0, self.stamina - 2.0)
        self.last_key = pygame.K_RIGHT
```

That's the **minimal fix**. No other part of the win/loss logic needs to change.

In one sentence for your lab/viva:

The bug is caused by `self.arm_position += 4.2` in both key handlers; since the player's winning condition is negative position, the increment moves the arm toward the computer, so changing `+= 4.2` to `-= 4.2` fixes the direction.

ok i changed both `+= 4.2` to `-= 4.2` like you said and now pressing the keys pulls the arm toward my side, so that seems to work. can you go through the whole `game_engine.py` once more and check if there are any other bugs i might have missed?

mainly i want to know:

does the computer still win properly if i don't press anything, and do i win at -100?

when i press R after the game ends, does everything actually reset (arm position, stamina, last key, winner)?

the keys stop working when stamina is 10 or below, but the README says below 10. is that a mistake?

stamina goes back up by 0.8 every frame and each press only costs 2, so i feel like i can never actually run out of stamina. is that a problem?

`update()` uses per-frame values, so would the game run faster or slower on a different computer?

if you find anything, tell me which function it's in, why it's a problem, and how to fix it. if something is fine, just say so.

Yes — after your `+= 4.2` → `-= 4.2` fix, I went through the whole `game_engine.py` logic. There are a few things worth fixing, but the win/loss and reset logic are basically correct.

1. Computer still wins if you don't press anything — Fine

Function: `update()`

Python

```
ai_variance = random.uniform(0.3, 1.0)
self.arm_position += self.ai_strength * ai_variance
```

The computer continuously adds a positive amount to `arm_position`.

So if you don't press anything:

Plain text

0 → +0.3-ish → +1 → +2 → ... → +100

Eventually this runs:

Python

```
elif self.arm_position >= self.target_limit:
    self.winner = "COMPUTER"
    self.game_state = "GAME_OVER"
```

Since:

Python

```
self.target_limit = 100.0
```

Computer wins correctly at +100.

2. Player wins at -100 — Fine

Also in `update()`:

Python

```
if self.arm_position <= -self.target_limit:
    self.winner = "PLAYER"
    self.game_state = "GAME_OVER"
```

Since `target_limit = 100.0`:

Plain text

```
arm_position <= -100
```

means player wins.

Your new:

```
</> Python

self.arm_position -= 4.2
```

moves toward that negative threshold.

So this part is correct.

3. Pressing R resets everything — Fine

Function: reset()

You have:

```
</> Python

def reset(self):
    self.arm_position = 0.0
    self.stamina = 100.0
    self.last_key = None
    self.winner = None
    self.game_state = "PLAYING"
```

This resets exactly the things you asked about:

Variable	Reset value	Correct?
arm_position	0.0	✓
stamina	100.0	✓
last_key	None	✓
winner	None	✓
game_state	"PLAYING"	✓

And `handle_event()` correctly calls it:

</> Python

```
if self.game_state != "PLAYING":  
    if event.type == pygame.KEYDOWN and event.key == pygame.K_r:  
        self.reset()
```

No bug here.

4. Stamina threshold says 10, README says below 10 — Minor inconsistency

Function: `handle_event()`

You currently have:

</> Python

```
if self.stamina <= 10:  
    return
```

That means the player cannot press at exactly 10 .

But your README says:

falling below 10 stamina temporarily halts input

"Below 10" technically means:

</> Python

```
self.stamina < 10
```

Minimal fix

If you want the code to exactly match the README:

</> Python

```
if self.stamina < 10:  
    return
```

However, there's a more important stamina issue...

5. Stamina probably won't run out — Yes, this is a real gameplay problem

Functions: `handle_event()` and `update()`

Each accepted key press costs:

</> Python

```
self.stamina -= 2.0
```

But every single frame you recover:

</> Python

```
self.stamina += 0.8
```

At 60 FPS, that's potentially:

</> Plain text

```
0.8 × 60 = 48 stamina/second
```

Meanwhile, even if you're pressing extremely rapidly, you're losing only:

</> Plain text

```
2 stamina per accepted keypress
```

And there's another important detail:

</> Python

```
if self.last_key != pygame.K_LEFT:
```

and

</> Python

```
if self.last_key != pygame.K_RIGHT:
```

means the same key can't repeatedly consume stamina. You have to alternate.

So yes: **the recovery is extremely fast compared with the cost.**

In normal gameplay, stamina will tend to remain high rather than meaningfully depleting.

Why this matters

Your README says:

Rapid pushing drains stamina; falling below 10 stamina temporarily halts input until it recovers.

But with `0.8` recovery **every frame**, stamina doesn't behave much like an exhaustion mechanic.

Minimal fix

Reduce the per-frame recovery substantially. For example:

`</>` Python

```
self.stamina = min(self.max_stamina, self.stamina + 0.08)
```

instead of:

`</>` Python

```
self.stamina = min(self.max_stamina, self.stamina + 0.8)
```

At 60 FPS, `0.08` gives approximately:

`</>` Plain text

$$0.08 \times 60 = 4.8 \text{ stamina/second}$$

That's much more reasonable.

But: this leads directly to your next issue.

6. `update()` is frame-rate dependent — Yes, this is a real bug/design problem

Function: `update()`

You currently do:

`</>` Python

```
self.arm_position += self.ai_strength * ai_variance
```

and:

```
</> Python
```

```
self.stamina += 0.8
```

These happen **once per frame**.

Your game runs at:

```
</> Python
```

```
FPS = 60
```

So at 60 FPS, the AI applies its force 60 times per second.

If the game somehow runs at 30 FPS, it applies it only 30 times per second.

Likewise, stamina recovery changes with FPS.

Why that's a problem

Imagine:

```
</> Plain text
```

```
60 FPS → 60 updates/sec
```

```
30 FPS → 30 updates/sec
```

At 30 FPS, the AI's effective movement per real-world second is roughly half.

So the game mechanics depend on how many frames the computer manages to process.

Better approach

Use elapsed time (`dt`) and make your rates **per second**.

For example, `main.py` could get the frame time:

```
</> Python
```

```
dt = clock.tick(FPS) / 1000.0
```

Then:

```
</> Python
```

```
engine.update(dt)
```

And `update()` becomes:

```
</> Python
```

```
def update(self, dt):
    if self.game_state != "PLAYING":
        return

    ai_variance = random.uniform(0.3, 1.0)
    self.arm_position += self.ai_strength * ai_variance * dt

    if self.stamina < self.max_stamina:
        self.stamina = min(
            self.max_stamina,
            self.stamina + 4.8 * dt
        )

    if self.arm_position <= -self.target_limit:
        self.winner = "PLAYER"
        self.game_state = "GAME_OVER"
    elif self.arm_position >= self.target_limit:
        self.winner = "COMPUTER"
        self.game_state = "GAME_OVER"
```

This makes the values represent **movement/recovery per second** instead of per frame.

One more thing: your key press itself is event-based

Your player movement:

```
</> Python
```

```
self.arm_position -= 4.2
```

happens when a `KEYDOWN` event occurs, rather than once every frame.

That's actually good for your button-mashing mechanic because you're measuring **actual key presses**, not holding a key down and getting movement every frame.

So I would **not** change that part just to make it frame-rate based.

Overall verdict

Area	Status
Left/Right direction	✓ Fixed
Player wins at -100	✓ Correct
Computer wins at +100	✓ Correct
Computer pushes automatically	✓ Correct
R resets arm	✓
R resets stamina	✓
R resets last key	✓
R resets winner/game state	✓
≤ 10 vs README "below 10"	⚠ Minor mismatch
Stamina recovery	⚠ Too fast
Frame-rate dependence	⚠ Real issue

What I'd fix for the lab

If you're trying to make the game **fully functional**, I would make these changes:

1. Match the README exactly:

Python

```
if self.stamina < 10:
```

2. Slow stamina recovery:

Python

```
self.stamina = min(self.max_stamina, self.stamina + 0.08)
```

3. Ideally make `update()` time-based using `dt`.

The third is the more technically correct solution, especially if your teacher asks "**Why is using 0.8 per frame problematic?**" You can say:

"Because the game logic is tied to frame count rather than elapsed time, the AI force and stamina recovery can change depending on the frame rate. Using delta time makes the game behavior consistent across different systems."

And importantly, **Tasks 2–4 haven't been implemented yet** in this file. `ai_strength` is still constant, there is no AI surge/cooldown state, no exhaustion warning, and no counter-surge bonus. Those are separate feature tasks rather than bugs in the existing code.

0.08 is making loose my stamina much earlier and never letting mw win so i prefer changing it to 0.3 , i sthat fine

Yes, **0.3 is a perfectly reasonable choice** for this game. 👍

With:

Python

```
self.stamina = min(self.max_stamina, self.stamina + 0.3)
```

at 60 FPS, you're recovering:

Plain text

```
0.3 × 60 = 18 stamina/second
```

Your pressing costs **2 stamina per accepted alternating keypress**, so you'll actually feel the stamina drain, but you won't become exhausted almost immediately like with 0.08 .

I would use

Python

```
if self.stamina < self.max_stamina:  
    self.stamina = min(self.max_stamina, self.stamina + 0.3)
```

And if you're keeping the README wording "**below 10**", use:

Python

```
if self.stamina < 10:
```

```
return
```

So your current balance becomes:

- **0.3 recovery/frame** → reasonable
- **2.0 cost/keypress** → meaningful drain
- **10 stamina threshold** → temporary exhaustion
- **100 max stamina** → enough room for sustained button mashing

One caveat: **0.3 is still frame-rate dependent**, which is fine for this lab if you're prioritizing gameplay balance. If you later implement the `dt` approach, we'd tune the equivalent *per-second* recovery to preserve this same feel.

task 2 works, the computer surges then gets tired. now task 3:
when stamina is low the keys stop working but nothing tells me why. can you add a visual warning, like a flashing red stamina bar, an "EXHAUSTED!" label, and the hand shaking a bit? also the lockout right now ends the moment stamina hits 10 again so it'd flicker, so can you make it stay locked until stamina recovers a bit more? show me the full changed code

Yes. For Task 3, I'd make the exhaustion state a **real state** rather than just checking `stamina < 10` every frame.

That gives you:

-  **Flashing red stamina bar**
- **EXHAUSTED!** label
- Slight **hand shake**
- Input locks when stamina drops below 10
- Input **doesn't immediately unlock at 10**; it stays locked until, for example, **25 stamina**
- Once recovered to 25, the exhaustion state clears

One important thing: **you said Task 2 is already working**, but I don't have your modified Task 2 `game_engine.py` in this message. I don't want to give you a "full changed code" that accidentally removes your AI surge/cooldown implementation.

Please paste your **current `game_engine.py` after Task 2** here. I'll then return the **complete file**, keeping all your Task 2 code exactly as-is and adding only the Task 3 changes.

```
import math
import random
import pygame
```

```
class GameEngine:

    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        self.stamina = 100.0
        self.max_stamina = 100.0

        self.winner = None
        self.game_state = "PLAYING"
        self.ai_strength = 0.35

        # Task 2: AI surge cycle BUILDING -> SURGING ->
        EXHAUSTED -> BUILDING
        self.ai_state = "BUILDING"
        self.ai_state_start = pygame.time.get_ticks()
        self.ai_build_ms = 4000
        self.ai_surge_ms = 1500
        self.ai_exhaust_ms = 2000
        self.ai_multipliers = {"BUILDING": 1.0, "SURGING": 2.0,
                                "EXHAUSTED": 0.3}

        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)

    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:
                self.reset()
                return

        if event.type == pygame.KEYDOWN:
            if self.stamina < 10:
```

```
        return

    if event.key == pygame.K_LEFT:
        if self.last_key != pygame.K_LEFT:
            self.arm_position -= 4.2
            self.stamina = max(0.0, self.stamina - 2.0)
            self.last_key = pygame.K_LEFT
    elif event.key == pygame.K_RIGHT:
        if self.last_key != pygame.K_RIGHT:
            self.arm_position -= 4.2
            self.stamina = max(0.0, self.stamina - 2.0)
            self.last_key = pygame.K_RIGHT

    def update(self):
        if self.game_state != "PLAYING":
            return

        # Task 2: advance the AI surge state machine using real
        # elapsed time
        now = pygame.time.get_ticks()
        elapsed = now - self.ai_state_start
        if self.ai_state == "BUILDING" and elapsed >=
self.ai_build_ms:
            self.ai_state, self.ai_state_start = "SURGING", now
        elif self.ai_state == "SURGING" and elapsed >=
self.ai_surge_ms:
            self.ai_state, self.ai_state_start = "EXHAUSTED", now
        elif self.ai_state == "EXHAUSTED" and elapsed >=
self.ai_exhaust_ms:
            self.ai_state, self.ai_state_start = "BUILDING", now

        ai_variance = random.uniform(0.3, 1.0)
        multiplier = self.ai_multipliers[self.ai_state]
        self.arm_position += self.ai_strength * ai_variance * multiplier

        if self.stamina < self.max_stamina:
            self.stamina = min(self.max_stamina, self.stamina + 0.3)

        if self.arm_position <= -self.target_limit:
            self.winner = "PLAYER"
```

```
        self.game_state = "GAME_OVER"
    elif self.arm_position >= self.target_limit:
        self.winner = "COMPUTER"
        self.game_state = "GAME_OVER"

    def reset(self):
        self.arm_position = 0.0
        self.stamina = 100.0
        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"
        self.ai_state = "BUILDING"
        self.ai_state_start = pygame.time.get_ticks()

    def render(self, screen):
        screen.fill((25, 28, 35))

        title_surf = self.font_big.render("ARM WRESTLE
SHOWDOWN", True, (240, 240, 240))
        screen.blit(title_surf, (self.width // 2 - title_surf.get_width() //
2, 12))

        player_header = self.font_med.render("PLAYER", True, (80,
160, 255))
        computer_header = self.font_med.render("COMPUTER", True,
(255, 100, 80))
        screen.blit(player_header, (60, 55))
        screen.blit(computer_header, (self.width - 150, 55))

        # Task 2: show the AI's current state under its header
        if self.ai_state == "SURGING":
            tag = self.font_med.render("SURGE!", True, (255, 200, 60))
            screen.blit(tag, (self.width - 150, 80))
        elif self.ai_state == "EXHAUSTED":
            tag = self.font_med.render("tired...", True, (150, 150, 170))
            screen.blit(tag, (self.width - 150, 80))

        table_rect = pygame.Rect(40, 100, self.width - 80, 310)
        pygame.draw.rect(screen, (110, 50, 15), table_rect,
border_radius=14)
```

```
pygame.draw.rect(screen, (70, 30, 8), table_rect, width=5,
border_radius=14)

pygame.draw.line(screen, (45, 18, 4), (self.width // 2, 100),
(self.width // 2, 410), 4)

offset_x = (self.arm_position / self.target_limit) * 95
hand_x = (self.width // 2) + int(offset_x)
hand_y = 235

p_shoulder = (70, 330)
p_elbow = (140, 215)
c_shoulder = (self.width - 70, 330)
c_elbow = (self.width - 140, 215)

pygame.draw.line(screen, (200, 145, 110), p_shoulder,
p_elbow, 32)
pygame.draw.line(screen, (215, 160, 125), p_elbow, (hand_x,
hand_y), 26)
pygame.draw.circle(screen, (185, 130, 95), p_elbow, 18)

pygame.draw.line(screen, (170, 110, 85), c_shoulder, c_elbow,
32)
pygame.draw.line(screen, (185, 125, 95), c_elbow, (hand_x,
hand_y), 26)
pygame.draw.circle(screen, (150, 95, 70), c_elbow, 18)

pygame.draw.circle(screen, (225, 175, 140), (hand_x, hand_y),
24)
pygame.draw.circle(screen, (160, 115, 85), (hand_x, hand_y),
24, width=3)

stamina_label = self.font_med.render("STAMINA", True, (220,
220, 220))
screen.blit(stamina_label, (40, 445))

stamina_bg = pygame.Rect(140, 448, 240, 22)
stamina_fill = pygame.Rect(140, 448, int(240 * (self.stamina /
self.max_stamina)), 22)
pygame.draw.rect(screen, (45, 50, 60), stamina_bg,
```

```

border_radius=6)
    bar_color = (60, 210, 100) if self.stamina > 25 else (220, 60,
60)
    pygame.draw.rect(screen, bar_color, stamina_fill,
border_radius=6)

    if self.game_state == "GAME_OVER":
        overlay = pygame.Surface((self.width, self.height),
pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 200))
        screen.blit(overlay, (0, 0))

        win_text = "PLAYER WINS THE MATCH!" if self.winner ==
"PLAYER" else "COMPUTER WINS!"
        color = (80, 240, 100) if self.winner == "PLAYER" else (240,
80, 80)
        text_surf = self.font_big.render(win_text, True, color)
        screen.blit(
            text_surf,
            (self.width // 2 - text_surf.get_width() // 2, self.height // 2
- 45)
        )

        restart_surf = self.font_med.render(
            "Press [R] to Rematch", True, (240, 240, 240)
        )
        screen.blit(
            restart_surf,
            (self.width // 2 - restart_surf.get_width() // 2, self.height
// 2 + 10)
        )

```

Perfect — your Task 2 code is clear. I'll keep the **AI surge system unchanged** and add Task 3 on top of it.

The important change is a new `self.player_exhausted` state:

- Stamina falls **below 10** → exhaustion activates.
- While exhausted → Left/Right inputs are ignored.
- Stamina reaching **10 does NOT unlock** the player.
- It stays locked until stamina reaches **25**.

- While exhausted:
 - stamina bar flashes red
 - EXHAUSTED! appears
 - player's hand shakes slightly
- reset() clears the exhaustion state.

Here is the **full updated game_engine.py** :

:

Task 3 — Updated game_engine.py



```
import math
import random
import pygame

class GameEngine:

    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        self.stamina = 100.0
        self.max_stamina = 100.0

        # Task 3: player exhaustion thresholds
        self.exhaustion_threshold = 10.0
        self.exhaustion_recovery = 25.0
        self.player_exhausted = False

        self.winner = None
        self.game_state = "PLAYING"
        self.ai_strength = 0.35

        # Task 2: AI surge cycle
        # BUILDING -> SURGING -> EXHAUSTED -> BUILDING
```

```
self.ai_state = "BUILDING"
self.ai_state_start = pygame.time.get_ticks()
self.ai_build_ms = 4000
self.ai_surge_ms = 1500
self.ai_exhaust_ms = 2000
self.ai_multipliers = {
    "BUILDING": 1.0,
    "SURGING": 2.0,
    "EXHAUSTED": 0.3
}

self.font_big = pygame.font.SysFont(None, 44)
self.font_med = pygame.font.SysFont(None, 26)

def handle_event(self, event):
    if self.game_state != "PLAYING":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if event.type == pygame.KEYDOWN:

        # Task 3: block player input while exhausted
        if self.player_exhausted:
            return

        if event.key == pygame.K_LEFT:
            if self.last_key != pygame.K_LEFT:
                self.arm_position -= 4.2
                self.stamina = max(0.0, self.stamina - 2.0)
                self.last_key = pygame.K_LEFT

        elif event.key == pygame.K_RIGHT:
            if self.last_key != pygame.K_RIGHT:
                self.arm_position -= 4.2
                self.stamina = max(0.0, self.stamina - 2.0)
                self.last_key = pygame.K_RIGHT

def update(self):
    if self.game_state != "PLAYING":
```

```

        return

# -----
# Task 2: advance the AI surge state machine using
# real elapsed time
# -----
now = pygame.time.get_ticks()
elapsed = now - self.ai_state_start

if self.ai_state == "BUILDING" and elapsed >= self.ai_build_ms:
    self.ai_state, self.ai_state_start = "SURGING", now

elif self.ai_state == "SURGING" and elapsed >= self.ai_surge_ms:
    self.ai_state, self.ai_state_start = "EXHAUSTED", now

elif self.ai_state == "EXHAUSTED" and elapsed >= self.ai_exhaust_ms:
    self.ai_state, self.ai_state_start = "BUILDING", now

# -----
# Task 2: apply AI pressure
# -----
ai_variance = random.uniform(0.3, 1.0)
multiplier = self.ai_multipliers[self.ai_state]
self.arm_position += self.ai_strength * ai_variance * multiplier

# -----
# Stamina recovery
# -----
if self.stamina < self.max_stamina:
    self.stamina = min(
        self.max_stamina,
        self.stamina + 0.3
    )

# -----
# Task 3: exhaustion state
#
# Enter exhaustion below 10.
# Do NOT leave exhaustion until stamina reaches 25.
# -----

```

```
    if not self.player_exhausted and self.stamina < self.exhaustion_thre
        self.player_exhausted = True
        self.last_key = None

    elif self.player_exhausted and self.stamina >= self.exhaustion_recov
        self.player_exhausted = False

    # -----
    # Win / loss detection
    # -----
    if self.arm_position <= -self.target_limit:
        self.winner = "PLAYER"
        self.game_state = "GAME_OVER"

    elif self.arm_position >= self.target_limit:
        self.winner = "COMPUTER"
        self.game_state = "GAME_OVER"

def reset(self):
    self.arm_position = 0.0
    self.stamina = 100.0
    self.last_key = None

    # Task 3: reset exhaustion state
    self.player_exhausted = False

    self.winner = None
    self.game_state = "PLAYING"

    # Task 2: reset AI surge cycle
    self.ai_state = "BUILDING"
    self.ai_state_start = pygame.time.get_ticks()

def render(self, screen):
    screen.fill((25, 28, 35))

    title_surf = self.font_big.render(
        "ARM WRESTLE SHOWDOWN",
        True,
        (240, 240, 240)
```

```
)
screen.blit(
    title_surf,
    (self.width // 2 - title_surf.get_width() // 2, 12)
)

player_header = self.font_med.render(
    "PLAYER",
    True,
    (80, 160, 255)
)

computer_header = self.font_med.render(
    "COMPUTER",
    True,
    (255, 100, 80)
)

screen.blit(player_header, (60, 55))
screen.blit(computer_header, (self.width - 150, 55))

# -----
# Task 2: show AI state
# -----
if self.ai_state == "SURGING":
    tag = self.font_med.render(
        "SURGE!",
        True,
        (255, 200, 60)
    )
    screen.blit(tag, (self.width - 150, 80))

elif self.ai_state == "EXHAUSTED":
    tag = self.font_med.render(
        "tired...",
        True,
        (150, 150, 170)
    )
    screen.blit(tag, (self.width - 150, 80))

# -----
```

```
# Table
# -----
table_rect = pygame.Rect(
    40,
    100,
    self.width - 80,
    310
)

pygame.draw.rect(
    screen,
    (110, 50, 15),
    table_rect,
    border_radius=14
)

pygame.draw.rect(
    screen,
    (70, 30, 8),
    table_rect,
    width=5,
    border_radius=14
)

pygame.draw.line(
    screen,
    (45, 18, 4),
    (self.width // 2, 100),
    (self.width // 2, 410),
    4
)

# -----
# Hand position
# -----
offset_x = (
    self.arm_position / self.target_limit
) * 95

hand_x = (self.width // 2) + int(offset_x)
```

```
hand_y = 235

# -----
# Task 3: shake the hand while exhausted
# -----
if self.player_exhausted:
    shake_time = pygame.time.get_ticks() / 70.0

    shake_x = int(math.sin(shake_time) * 4)
    shake_y = int(math.cos(shake_time * 1.3) * 3)

    hand_x += shake_x
    hand_y += shake_y

p_shoulder = (70, 330)
p_elbow = (140, 215)

c_shoulder = (self.width - 70, 330)
c_elbow = (self.width - 140, 215)

pygame.draw.line(
    screen,
    (200, 145, 110),
    p_shoulder,
    p_elbow,
    32
)

pygame.draw.line(
    screen,
    (215, 160, 125),
    p_elbow,
    (hand_x, hand_y),
    26
)

pygame.draw.circle(
    screen,
    (185, 130, 95),
    p_elbow,
```

```
        18
    )

    pygame.draw.line(
        screen,
        (170, 110, 85),
        c_shoulder,
        c_elbow,
        32
    )

    pygame.draw.line(
        screen,
        (185, 125, 95),
        c_elbow,
        (hand_x, hand_y),
        26
    )

    pygame.draw.circle(
        screen,
        (150, 95, 70),
        c_elbow,
        18
    )

    pygame.draw.circle(
        screen,
        (225, 175, 140),
        (hand_x, hand_y),
        24
    )

    pygame.draw.circle(
        screen,
        (160, 115, 85),
        (hand_x, hand_y),
        24,
        width=3
    )
```

```
# -----  
# Stamina  
# -----  
stamina_label = self.font_med.render(  
    "STAMINA",  
    True,  
    (220, 220, 220)  
)  
  
screen.blit(stamina_label, (40, 445))  
  
stamina_bg = pygame.Rect(  
    140,  
    448,  
    240,  
    22  
)  
  
stamina_fill = pygame.Rect(  
    140,  
    448,  
    int(240 * (self.stamina / self.max_stamina)),  
    22  
)  
  
pygame.draw.rect(  
    screen,  
    (45, 50, 60),  
    stamina_bg,  
    border_radius=6  
)  
  
# -----  
# Task 3: flashing red stamina bar during exhaustion  
# -----  
if self.player_exhausted:  
    flash_on = (pygame.time.get_ticks() // 250) % 2 == 0  
  
    if flash_on:
```

```
        bar_color = (255, 40, 40)
    else:
        bar_color = (120, 20, 20)

else:
    bar_color = (
        (60, 210, 100)
        if self.stamina > 25
        else (220, 60, 60)
    )

pygame.draw.rect(
    screen,
    bar_color,
    stamina_fill,
    border_radius=6
)

# -----
# Task 3: exhaustion warning
# -----
if self.player_exhausted:
    exhausted_text = self.font_big.render(
        "EXHAUSTED!",
        True,
        (255, 60, 60)
    )

    screen.blit(
        exhausted_text,
        (
            self.width // 2 - exhausted_text.get_width() // 2,
            465
        )
    )

# -----
# Game over screen
# -----
if self.game_state == "GAME_OVER":
```

```
overlay = pygame.Surface(
    (self.width, self.height),
    pygame.SRCALPHA
)

overlay.fill((0, 0, 0, 200))
screen.blit(overlay, (0, 0))

win_text = (
    "PLAYER WINS THE MATCH!"
    if self.winner == "PLAYER"
    else "COMPUTER WINS!"
)

color = (
    (80, 240, 100)
    if self.winner == "PLAYER"
    else (240, 80, 80)
)

text_surf = self.font_big.render(
    win_text,
    True,
    color
)

screen.blit(
    text_surf,
    (
        self.width // 2 - text_surf.get_width() // 2,
        self.height // 2 - 45
    )
)

restart_surf = self.font_med.render(
    "Press [R] to Rematch",
    True,
    (240, 240, 240)
)
```

```

        screen.blit(
            restart_surf,
            (
                self.width // 2 - restart_surf.get_width() // 2,
                self.height // 2 + 10
            )
        )

```

What changed from your Task 2 version

Only the Task 3 additions are new:

1. Exhaustion state

```

self.exhaustion_threshold = 10.0
self.exhaustion_recovery = 25.0
self.player_exhausted = False

```

2. Input lock

Instead of simply doing `stamina < 10` inside `handle_event()`, it checks:

```

if self.player_exhausted:
    return

```

3. Enter/exit exhaustion

```

if not self.player_exhausted and self.stamina < 10:
    self.player_exhausted = True
    self.last_key = None

elif self.player_exhausted and self.stamina >= 25:
    self.player_exhausted = False

```

So there is now a **hysteresis gap**:

```

100
|
| normal
↓

```

10 — EXHAUSTION STARTS

|

| locked

|

25 — EXHAUSTION ENDS

|

↓

normal input

This prevents the annoying 10 → 10.3 → unlock → 9.9 → lock flickering behavior.

4. Flashing bar

It alternates between dark and bright red every 250 ms while exhausted.

5. Hand shake

The hand gets a small smooth oscillating offset while exhausted, rather than random movement every frame.

One thing to test

Try deliberately button-mashing until the stamina goes below 10. You should see:

stamina < 10 → EXHAUSTED! → flashing red bar → shaking hand → keys don't work → stamina reaches 25 → everything returns to normal.

Your Task 2 AI surge/cooldown remains intact.

</> Python

```
import math
import random
import pygame
```

```
class GameEngine:
```

```
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None
```

```

self.stamina = 100.0
self.max_stamina = 100.0

# Task 3: player exhaustion thresholds
self.exhaustion_threshold = 10.0

```

What changed from your Task 2 version

Only the Task 3 additions are new:

1. Exhaustion state

</> Python

```

self.exhaustion_threshold = 10.0
self.exhaustion_recovery = 25.0
self.player_exhausted = False

```

2. Input lock

Instead of simply doing `stamina < 10` inside `handle_event()`, it checks:

</> Python

```

if self.player_exhausted:
    return

```

3. Enter/exit exhaustion

</> Python

```

if not self.player_exhausted and self.stamina < 10:
    self.player_exhausted = True
    self.last_key = None

elif self.player_exhausted and self.stamina >= 25:
    self.player_exhausted = False

```

So there is now a **hysteresis gap**:

</> Plain text

```

100
|
| normal
↓
10 — EXHAUSTION STARTS

```

```

|
| locked
|
25 — EXHAUSTION ENDS
|
↓
normal input

```

This prevents the annoying **10** → **10.3** → **unlock** → **9.9** → **lock** flickering behavior.

4. Flashing bar

It alternates between dark and bright red every 250 ms while exhausted.

5. Hand shake

The hand gets a small smooth oscillating offset while exhausted, rather than random movement every frame.

One thing to test

Try deliberately button-mashing until the stamina goes below 10. You should see:

stamina < 10 → **EXHAUSTED!** → **flashing red bar** → **shaking hand** → **keys don't work** → **stamina reaches 25** → **everything returns to normal**.

Your Task 2 AI surge/cooldown remains intact.

i wish to add timer in the game when the game starts and resets accordingly

Yes — a timer fits nicely. I'd make it a **match timer** that:

- Starts when the game first starts.
- Stops when `GAME_OVER` is reached.
- Resets to `00:00` when you press **R**.
- Starts again immediately after the rematch.
- Shows the elapsed time near the top of the game.
- Uses `pygame.time.get_ticks()`, so it measures real elapsed time rather than frames.

Since your Task 2 already uses `pygame.time.get_ticks()`, this fits your existing code well.

Changes you need

In `__init__()` add:

```
</> Python
```

```
self.game_start_time = pygame.time.get_ticks()
self.game_end_time = None
```

In `reset()` :

</> Python

```
self.game_start_time = pygame.time.get_ticks()
self.game_end_time = None
```

In `update()` , when someone wins, save the end time:

</> Python

```
if self.arm_position <= -self.target_limit:
    self.winner = "PLAYER"
    self.game_state = "GAME_OVER"
    self.game_end_time = pygame.time.get_ticks()

elif self.arm_position >= self.target_limit:
    self.winner = "COMPUTER"
    self.game_state = "GAME_OVER"
    self.game_end_time = pygame.time.get_ticks()
```

Then in `render()` , calculate the elapsed time:

</> Python

```
if self.game_end_time is not None:
    elapsed_ms = self.game_end_time - self.game_start_time
else:
    elapsed_ms = pygame.time.get_ticks() - self.game_start_time

elapsed_seconds = elapsed_ms // 1000
minutes = elapsed_seconds // 60
seconds = elapsed_seconds % 60

timer_text = self.font_med.render(
    f"TIME {minutes:02d}:{seconds:02d}",
    True,
    (240, 240, 240)
)

screen.blit(
    timer_text,
    (self.width // 2 - timer_text.get_width() // 2, 55)
)
```

So you'll see something like:

</> Plain text

PLAYER

TIME 00:17

COMPUTER

And when the match ends at 00:23, it stays at **00:23** rather than continuing to count while the Game Over screen is displayed.

One small improvement

Because you already use `pygame.time.get_ticks()` for your **AI surge cycle**, I recommend this approach rather than adding another timer or using `clock.tick()` for the match timer. It keeps the timer independent of FPS and consistent with your Task 2 implementation.

If you want, I can:

- ↳ Add game timer start and reset
- ↳ Display elapsed match time on screen
- ↳ Pause timer when game is over and resume on reset